1. Definición de arquitectura y stack tecnológico

1.1. Arquitectura del sistema

Arquitectura seleccionada
Monolítica modular

El Sistema Integral de Gestión Digital del Patronato utilizará una arquitectura monolítica modular.

Esto significa que el sistema se desarrolla como una sola aplicación, pero su código se organiza en módulos independientes según las funciones del Patronato.

Cada módulo tendrá responsabilidades específicas, lo que facilita el desarrollo, mantenimiento y ampliación del sistema.

Ventajas de esta arquitectura:

- Centraliza la administración del sistema.
- Permite que los cuatro integrantes desarrollen módulos diferentes.
- Facilita el mantenimiento y la reutilización del código.
- Comparte una misma base de datos.
- Evita la complejidad inicial de implementar múltiples microservicios.

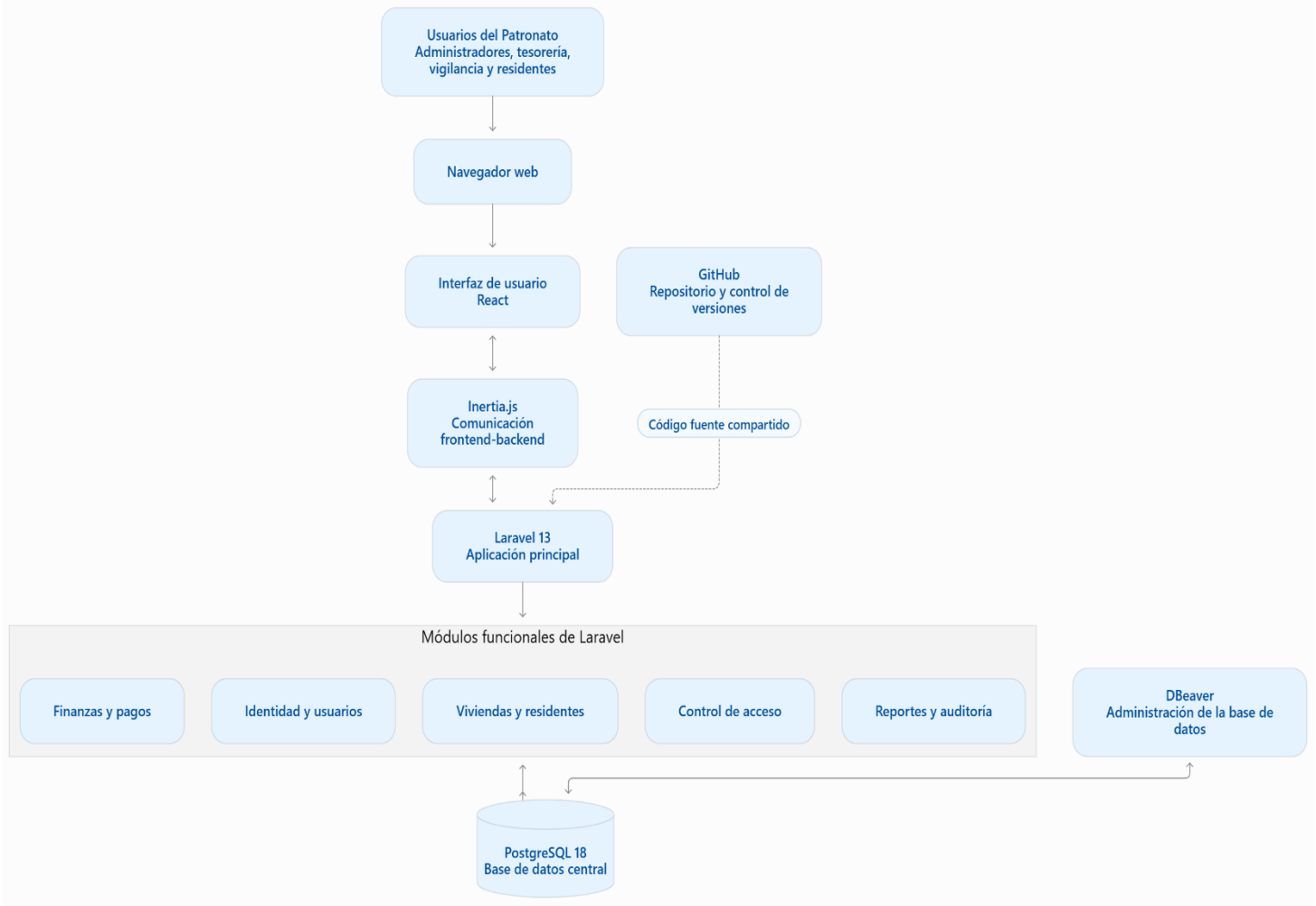
1.2. Stack de tecnologías

Tecnología	Función dentro del proyecto
PHP 8.4	Lenguaje de programación del backend.
Laravel 13	Framework para la lógica del sistema, rutas, controladores y seguridad.
Inertia.js	Comunicación entre Laravel y la interfaz de usuario.
React	Desarrollo de las interfaces visuales.
Node.js y npm	Administración de dependencias y compilación del frontend.
PostgreSQL 18	Sistema gestor de base de datos.
DBeaver	Herramienta gráfica para administrar y consultar la base de datos.
Composer	Administración de dependencias de PHP.
Git y GitHub	Control de versiones y trabajo colaborativo.
Visual Studio Code	Editor de código para desarrollar el proyecto.

Aclaración: DBeaver es el cliente de administración de PostgreSQL; el servidor de base de datos es PostgreSQL. Asimismo, Visual Studio Code es el editor que venimos utilizando, no Visual Studio IDE.

1.3. Diagrama de arquitectura

Este diagrama representa cómo se relacionan los componentes del sistema:



Explicación del funcionamiento:

1. El usuario ingresa al sistema desde un navegador web.
2. React presenta las pantallas y los formularios.
3. Inertia permite la comunicación entre React y Laravel.
4. Laravel procesa las solicitudes, aplica las reglas del negocio y organiza las funciones en módulos.
5. PostgreSQL almacena la información del sistema.
6. DBeaver permite consultar y administrar la base de datos.
7. GitHub permite que los cuatro integrantes compartan y controlen los cambios del código.

1.4. Organización modular propuesta

Los módulos funcionales se organizan de acuerdo con las responsabilidades del sistema:

Módulo	Funcionalidad principal
Identidad	Usuarios, autenticación y permisos.
Residenciales	Administración de residenciales y configuración.
Viviendas y residentes	Registro y administración de viviendas y habitantes.
Finanzas	Cuotas, cargos, pagos, ingresos y egresos.
Control de acceso	Vehículos, visitantes, carnets, stickers y QR.
Vigilancia	Guardias, turnos e incidencias.
Reportes y auditoría	Consultas, movimientos y seguimiento de operaciones.

Los módulos son una organización funcional propuesta a partir del alcance del proyecto. Su ubicación exacta en carpetas debe respetar la estructura que ya definió el equipo.

El proyecto utilizará una arquitectura monolítica modular, implementada mediante Laravel. Esta arquitectura permitirá desarrollar el sistema como una sola aplicación, organizada en módulos funcionales independientes.

Para el backend utilizaremos PHP y Laravel; para el frontend, React e Inertia.js; y para el almacenamiento de información, PostgreSQL. DBeaver será nuestra herramienta gráfica para administrar la base de datos.

El desarrollo colaborativo se realizará mediante Git y GitHub, permitiendo que los cuatro integrantes trabajemos en diferentes historias de usuario y compartamos los avances mediante el control de versiones.

2. Instalación y preparación del entorno de desarrollo

Esta parte indica como cada integrante pueda preparar su computadora.

2.1. Tecnologías que debe instalar cada integrante

Herramienta	Versión de referencia
Git	Versión estable disponible
PHP	8.4
Composer	2.x
Node.js	26.x
Laravel	13.x
PostgreSQL	18.x
DBeaver	Community Edition

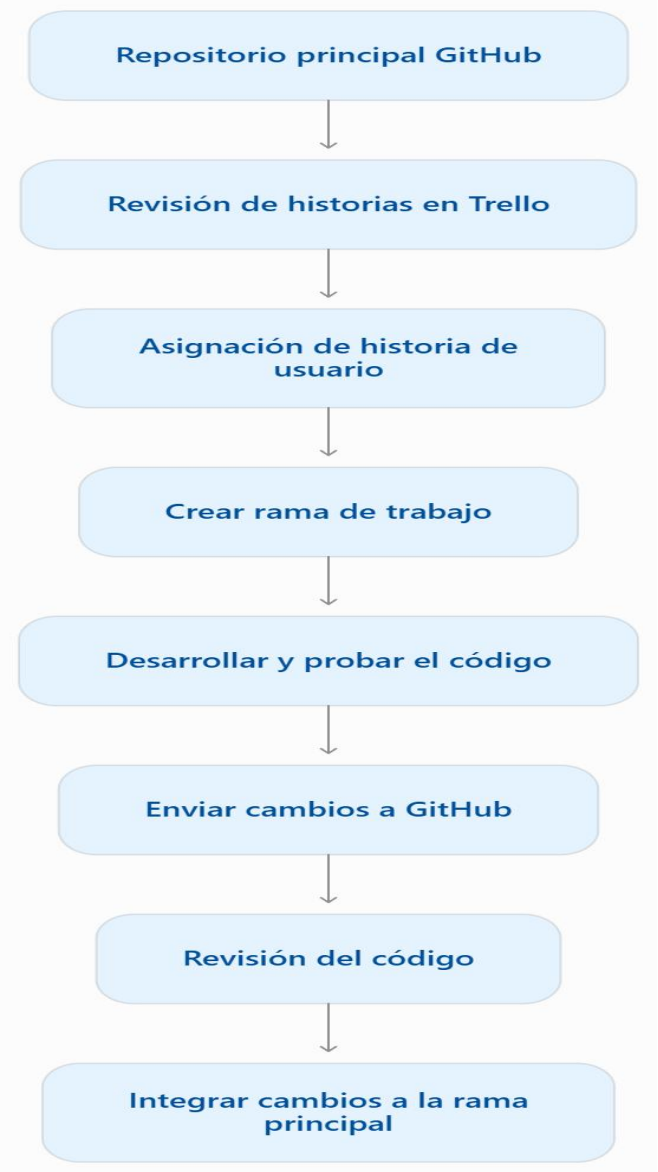
Visual Studio Code Versión estable

Cabe hacer notar que las versiones deben mantenerse compatibles con las dependencias y archivos de configuración del repositorio. No se debe actualizar una versión por cuenta propia si el equipo ha establecido otra.

3. Trabajo colaborativo de los cuatro integrantes

Para evitar conflictos en el desarrollo, se recomienda trabajar mediante historias de usuario y ramas de Git.

Flujo de trabajo propuesto:



4. Tecnologías a usar en la arquitectura

